VerifiedX

TECHNICAL SPECIFICATION

VERIFIED BITCOIN ON BASE

vBTC.b Technical Specification



Version 4 · Core testnet ee83f568

November 2026 · for integrators, auditors and institutional reviewers

VERSION 4 This document describes Version 4 of the vBTCb contract and the VerifiedX Core rules on testnet at commit ee83f568. Base Sepolia runs V4. **Base mainnet runs V3 until its proxy is upgraded**; §9 describes V3 and the upgrade. It replaces the earlier vBTC.b Technical Specification, which described Version 2 of the bridge.

ABSTRACT

vBTC.b carries vBTC from VerifiedX to Base as an 8-decimal ERC-20 token. A holder locks vBTC on VerifiedX; once the lock is buried 30 blocks deep, the contract's signers attest it and the vBTCb contract mints the same amount on Base. The contract has no owner: a mint and every change to it need two thirds of its validators and a majority of its casters, over EIP-712 typed messages, and code upgrades wait at least 48 hours in public. Burning vBTC.b returns vBTC on VerifiedX from the pool of locks, or pays Bitcoin from the vBTC V2 vaults behind those locks through a transaction that VerifiedX consensus fixes and the vaults' key holders sign. This document specifies those rules, their parameters and their trust assumptions.

APPLIES TO
vBTCb V4 (MESSAGE_VERSION 4)
Core testnet ee83f568

NETWORKS
Base Sepolia 84532 (V4)
Base mainnet 8453 (V3)

SIGNATURES
EIP-712 typed data
secp256k1 ECDSA

COMPANION DOCS
docs.verifiedx.io
vBTC · Bridging to Base

Contents

1	**Introduction** scope, version status, design properties	3
2	**System overview** roles, layers, thresholds at a glance	4
3	**The vBTCb contract** token, proxy, interface, constants	5
4	**Signer groups and thresholds**	7
5	**EIP-712 typed data**	7
6	**Minting** lock, burial, attestation, mintWithProof	9
7	**Exits** caster agreement, unlock to VerifiedX, exit to Bitcoin	11
8	**Governance** set changes, timelocked upgrades, cancellation	13
9	**Upgrading from V3**	15
10	**Transaction types**	16
11	**Parameters**	16
12	**Security model** assumptions, attacks and mitigations, limits	17
13	**Deployed addresses**	18
14	**Glossary and references**	18

What changed from the Version 2 edition

TOPIC	VERSION 2 EDITION	THIS EDITION (V4, CORE TESTNET)
Signed messages	Packed fields under the EIP-191 personal-message prefix.	EIP-712 typed data in the domain ("vBTCb", "4", chain id, contract), one type hash per action (§5).
Signers	Validators only.	Two groups: validators (3 to 50) and casters (3 to 10). A mint and every change need both (§4).
Upgrades	`upgradeWithValidatorApproval`, effective at once.	Proposed, then executable after `upgradeDelay()` (48 hours to 30 days), lapsing 14 days later (§8.4).
Validator removal	51% of validators (a simple majority up to 5).	The mint approval. A majority acts only through a timelocked, cancellable proposal (§8.6).
Validator addition	Signatures of the existing set.	Also each new validator's own signature over the same message (§8.2).
Bad signatures	A malformed signature reverts the call.	Skipped; only the count decides (§4.4).
Mint attestation	On the lock's confirmation.	Only once the lock has 30 blocks on top, from a validator on the network's chain (§6.3).
Exit to Bitcoin	A caster builds and FROST-signs the payment and records completion or failure (types 41, 42); adaptive signing threshold.	Consensus pins each vault's payment; key holders sign exactly that; Bitcoin settles it; failure by rule. Types 41 and 42 are refused for these exits (§7.5 to §7.8).

CONVENTIONS

"Must" states a rule that the contract or VerifiedX consensus enforces: a call that breaks it reverts on Base, and a transaction that breaks it is invalid on every VerifiedX node. **Policy** marks what VerifiedX Core nodes do that consensus does not enforce. *n* is the number of validators on the contract, *c* the number of casters on the contract and *k* the size of the VerifiedX caster committee. Amounts on Base are integers in satoshis (8 decimals). Heights are VerifiedX (VFX) block heights unless marked as Bitcoin or Base.

1 Introduction

vBTC.b is vBTC on Base. Each vBTC.b is minted against vBTC locked on VerifiedX, and the Bitcoin behind it stays in the vBTC V2 vault that held it. Burning vBTC.b on Base returns vBTC on VerifiedX or pays Bitcoin out of those vaults.

The bridge has two halves. On Base, the **vBTCb contract** holds the token and its rules: it mints only with signatures from two groups, its validators and its casters, and it has no owner. On VerifiedX, **consensus** records each lock, credits unlocks from the pool of locks, and fixes the Bitcoin transaction that pays each exit to Bitcoin, which the vault's key holders sign as they would any vBTC V2 withdrawal.

VERSION STATUS

This document describes vBTCb Version 4 (`MESSAGE_VERSION()` = 4) and the Core rules on testnet at commit ee83f568. Base Sepolia runs V4. Base mainnet runs V3 until its proxy is upgraded in place; §9 states what V3 does differently. Where this document and the source differ, the source governs.

The design aims for these properties, each specified in the sections that follow:

- **Two groups for everything.** A mint, a change to either signer list, a proposed upgrade and a proposed delay change each need two thirds of the validators and a majority of the casters.
- **One message type per action.** Every signature is EIP-712 typed data with its own type hash, bound to the chain and the contract, so no signature can be read as a different action.
- **No owner and no instant upgrade.** Code changes wait at least 48 hours after approval, either group's blocking minority can cancel them, and an approval lapses if it is not used.
- **Mints against buried locks only.** Validators sign a mint only for a lock in their own chain state with 30 blocks on top of it.
- **Exits to Bitcoin paid by rule.** The Bitcoin transaction for each vault's part of an exit is computed by every node; signers sign only that; Bitcoin's confirmation settles it.
- **Seats only for held keys.** A new validator or caster signs its own addition, so no seat goes to an address whose key nobody holds.

Out of scope: the vBTC V2 vaults themselves (key ceremony, Bitcoin attestation, withdrawals; see the vBTC V2 Technical Specification), vaults created before the attestation recording height, the shielded vBTC pool and wallet interfaces. vBTC cannot be bridged from an S3C vault.

2 System overview

2.1 Roles

ROLE	DEFINITION
Holder	Holds vBTC on VerifiedX or vBTC.b on Base. Locks its own vBTC; its node collects the mint signatures and submits the mint from a Base address derived from the holder's VFX key, which pays the Base gas. Burns vBTC.b from any Base address.
Contract validator	A Base address on the contract's validator list. Core keeps the list to registered public vBTC validators that hold the 5,000 VFX validator balance, each by the Base address of its own key (§8.2).
Contract caster	A Base address on the contract's caster list. Core keeps the list in step with the VerifiedX caster committee (§8.3). An address may be on both lists.
Caster committee	The VerifiedX casters named by the signed membership record for a height. Detects burns on Base, elects a handler per burn and signs votes; the handler submits unlocks (type 39) and exits (type 40).
Vault key holders	The validators holding FROST shares of a vault's key. A threshold of them signs exactly the Bitcoin transaction consensus pinned for that vault's part of an exit.
Every node	Any VerifiedX node. Verifies every lock, unlock, exit, pin and settlement from chain state.
Anyone on Base	May submit `mintWithProof` with valid signatures, and may execute an upgrade, delay change or removal once its delay has passed.

2.2 Layers

Holders — AUTHORIZATION
Sign their own lock on VerifiedX and their own burn on Base. Nothing else moves their vBTC or vBTC.b.

vBTCb on Base — ISSUANCE
ERC-20 behind a UUPS proxy. Mints against signatures from validators and casters, burns on request, and changes itself only through signed, nonce-ordered and (for code) timelocked messages.

VerifiedX consensus — LOCKS, POOL, EXITS
Holds every lock, the pool they form, every exit and its pinned Bitcoin transactions as chain state, identical on every node.

Bitcoin vaults — BACKING
The Bitcoin stays in vBTC V2 vaults at their Taproot addresses. An exit to Bitcoin is a vault transaction like a withdrawal.

Mint
Lock (type 37), 30 blocks, signatures, `mintWithProof`.

Unlock
Burn on Base, caster votes, pool unlock (type 39).

Exit to Bitcoin
Burn, caster votes, exit (type 40), pin, sign, Bitcoin.

Change
Typed admin messages, each at the contract's admin nonce.

2.3 Thresholds at a glance

"Both" means `⌈2n/3⌉` validators (never fewer than 2) **and** `⌊c/2⌋+1` casters, counted from one list of signatures. On a contract with no casters (a V3 proxy upgraded in place, §9) the caster part is zero.

ACTION	WHERE	SIGNATURES NEEDED
Mint	`mintWithProof`	Both
Add validators	`addValidatorBatch`	Both, and each new validator's own
Remove validators	`removeValidatorBatch`	Both
Change casters	`updateCasters`	Both, and each added caster's own
Propose an upgrade	`proposeUpgrade`	Both; executable after `upgradeDelay()`
Propose a delay change	`proposeUpgradeDelay`	Both; executable after the current delay
Propose a removal	`proposeRemoval`	`⌊n/2⌋+1` validators (at least 2) and `⌊c/2⌋+1` casters, or `⌈2n/3⌉` validators alone; executable after the delay
Cancel what is pending	`cancelPending`	`n − ⌈2n/3⌉ + 1` validators, or `c − (⌊c/2⌋+1) + 1` casters
Execute a matured proposal	`execute…`	None; anyone, from its eta until 14 days after
Unlock to VerifiedX	Type 39 on VerifiedX	`max(2, ⌊k/2⌋+1)` committee casters' bound votes
Exit to Bitcoin	Type 40 on VerifiedX	The same votes; the payment is then signed by the vault's key threshold

3 The vBTCb contract

3.1 Token

vBTCb is an OpenZeppelin `ERC20Upgradeable` (5.x) token. The deploy script initializes it with the name "Verified Bitcoin on Base" and the symbol "vBTC.b"; `decimals()` is 8, so one unit is one satoshi. Supply changes only by `mintWithProof` and by the two burn functions, which burn from the caller. There is no pause, no blocklist, no transfer fee and no allowance-based burn.

3.2 Proxy and storage

The implementation sits behind an ERC-1967 proxy using UUPS. Its constructor disables initializers, so the implementation contract itself cannot be initialized. `_authorizeUpgrade` reverts unless `executeUpgrade` set a one-call flag, so calling `upgradeToAndCall` directly always reverts; `executeUpgrade` passes it empty call data. V4's storage is appended after V3's last variable and taken from the reserved gap, which shrinks from 50 slots to 40; `pendingImplementation` shares V3's last slot.

3.3 No owner

The contract has no `Ownable`, no roles and no address with special rights; the deployer receives none. Every privileged action needs signatures (§4). Executing a proposal whose delay has passed needs none and is open to anyone.

3.4 Interface

FUNCTION	EFFECT AND CHECKS
`mintWithProof(to, amount, lockId, nonce, signatures)`	Mints `amount` to `to` (§6.5).
`burnForVfxExit(amount, vfxDestinationAddress)`	Burns from the caller; `amount > 0`, destination not empty. Emits `VfxExitBurned(burner, amount, vfxDestinationAddress, chainId)`.
`burnForBTCExit(amount, btcDestination)`	Burns from the caller; `amount > 0`, destination at least 26 bytes. Emits `BTCExitBurned(burner, amount, btcDestination, chainId)`.
`addValidatorBatch(newValidators, vfxBlockHeight, signatures, newValidatorSignatures)`	§8.2
`removeValidatorBatch(oldValidators, vfxBlockHeight, signatures)`	§8.2
`updateCasters(add, remove, vfxBlockHeight, signatures, newCasterSignatures)`	§8.3
`proposeUpgrade(newImplementation, signatures)` · `executeUpgrade()`	§8.4
`proposeUpgradeDelay(newDelay, signatures)` · `executeUpgradeDelay()`	§8.5
`proposeRemoval(oldValidators, oldCasters, vfxBlockHeight, signatures)` · `executeRemoval()`	§8.6
`cancelPending(signatures)`	§8.7
Views	`getValidators`, `getCasters`, `casterCount`, `requiredMintSignatures`, `requiredCasterSignatures`, `requiredVetoSignatures`, `requiredCasterVetoSignatures`, `upgradeDelay`, `adminNonce`, `isLockIdUsed`, `DOMAIN_SEPARATOR`, `MESSAGE_VERSION`, the pending proposal fields and every type hash.

The contract does not check that a destination is a valid VerifiedX or Bitcoin address. A burn whose destination VerifiedX consensus does not accept is never unlocked or paid (§12.4), so wallets must validate destinations before burning.

3.5 Constants

CONSTANT	VALUE	MEANING
`MIN_VALIDATORS` · `MAX_VALIDATORS`	3 · 50	Bounds of the validator list.
`MIN_CASTERS` · `MAX_CASTERS`	3 · 10	Bounds of the caster list once it has casters.
`MIN_REQUIRED_SIGNATURES`	2	Floor of the validator thresholds.
`DEFAULT_UPGRADE_DELAY`	48 hours	The delay until changed, and its minimum.
`MAX_UPGRADE_DELAY`	30 days	Largest delay that can be set.
`PROPOSAL_GRACE_PERIOD`	14 days	A proposal lapses this long after it became executable.
`MESSAGE_VERSION`	4	Read by Core to choose the signature form (§9).
`MIN_VALIDATORS_FOR_HIGH_THRESHOLD`	5	Kept from V3; no longer used.

`requiredRemoveSignatures` is still stored (`max(2, ⌊n/2⌋+1)` after any validator change) but V4 reads no threshold from it: `proposeRemoval` computes the majority from the current count.

4 Signer groups and thresholds

4.1 Validators

The validator list holds 3 to 50 distinct, non-zero addresses. It is set by `initialize` and changed only by `addValidatorBatch`, `removeValidatorBatch` and `executeRemoval`; thresholds are recalculated after each change. A validator's Base address is the Ethereum-style address of its own VFX secp256k1 key, the key its vBTC validator registration carries.

4.2 Casters

The caster list holds 3 to 10 distinct, non-zero addresses, set by `initialize` and changed by `updateCasters` and `executeRemoval`. It can be empty only on a V3 proxy upgraded in place, until the first `updateCasters` (§9). A caster's Base address is derived from the public key in its caster committee record.

4.3 Thresholds

THRESHOLD	FORMULA	N = 3 · 4 · 5 · 7 · 10 · 50
Validator approval	`max(2, ⌊(2n+2)/3⌋) = ⌈2n/3⌉`	2 · 3 · 4 · 5 · 7 · 34
Validator majority	`max(2, ⌊n/2⌋+1)`	2 · 3 · 3 · 4 · 6 · 26
Validator veto	`n − approval + 1`	2 · 2 · 2 · 3 · 4 · 17

THRESHOLD	FORMULA	C = 3 · 4 · 5 · 10 (0 WHEN C = 0)
Caster majority	`⌊c/2⌋+1`	2 · 3 · 3 · 6
Caster veto	`c − majority + 1`	2 · 2 · 3 · 5

The veto is the blocking minority: the smallest number of a group whose refusal alone prevents a mint. It is what `cancelPending` requires (§8.7).

4.4 Counting signatures

Every check counts the same way, over one `bytes[]` list. For each signature in order, the contract:

1. recovers the signer with OpenZeppelin `ECDSA.tryRecover`. A signature that is not 65 bytes, has *s* in the upper half of the curve order or recovers nothing is **skipped**, not reverted on;
2. skips a signer on neither list, and a signer already counted;
3. counts the signer once toward the validators if it is a validator and once toward the casters if it is a caster: an address on both lists counts for both groups;
4. stops when both counts reach what the check asks.

The call reverts only when the counts fall short. A signer cannot be counted twice, and one bad or foreign signature cannot block a call that has enough good ones. Because an address on both lists satisfies both groups with one signature, the two-group rule gives independent protection only to the extent the lists hold different keys. The Base Sepolia proxy was initialized with the same three addresses as validators and casters (Core `deploy/README.md`).

5 EIP-712 typed data

5.1 Domain and digest

```
EIP712Domain(string name,string version,uint256 chainId,address verifyingContract)
name = "vBTCb"   version = "4"   chainId = block.chainid   verifyingContract = the proxy

digest = keccak256(0x19 0x01 ‖ domainSeparator ‖ structHash)
structHash = keccak256(typeHash ‖ encoded fields)
```

Signers sign the 32-byte digest directly with their secp256k1 key, producing a 65-byte (r, s, v) signature. No personal-message prefix is applied. Because the domain carries the chain id and the proxy address, a signature is valid on one contract on one chain only.

5.2 Message types

TYPE STRING (ITS KECCAK256 IS THE TYPE HASH)	USED BY
`Mint(address to,uint256 amount,string lockId,uint256 nonce)`	`mintWithProof`
`AddValidators(address[] validators,uint256 vfxBlockHeight,uint256 adminNonce)`	`addValidatorBatch`
`RemoveValidators(address[] validators,uint256 vfxBlockHeight,uint256 adminNonce)`	`removeValidatorBatch`
`UpdateCasters(address[] add,address[] remove,uint256 vfxBlockHeight,uint256 adminNonce)`	`updateCasters`
`ProposeRemoval(address[] validators,address[] casters,uint256 vfxBlockHeight,uint256 adminNonce)`	`proposeRemoval`
`ProposeUpgrade(address implementation,uint256 adminNonce)`	`proposeUpgrade`
`ProposeUpgradeDelay(uint256 delay,uint256 adminNonce)`	`proposeUpgradeDelay`
`CancelPending(uint256 adminNonce)`	`cancelPending`

Field encoding follows EIP-712: an `address` or `uint256` is one 32-byte word; `string lockId` is `keccak256(bytes(lockId))`; an `address[]` is the keccak256 of its elements' 32-byte words in order (the contract's `keccak256(abi.encodePacked(list))`, which pads array elements to 32 bytes). Core computes the same digests in `VbtcbTypedData.cs`.

5.3 Why one signature cannot serve as another

- **Different actions.** Each struct hash begins with its own type hash, so the digest of one action never equals the digest of another. In V3 the packed admin messages could also be read as `mintWithProof` arguments (§9).
- **Different contracts or chains.** The domain separator binds the chain id and the proxy address.
- **The same action again.** Every admin message carries `adminNonce`, which each admin call increments (§8.1): a signature set is valid at one nonce only, and only until the next admin action. A mint is used once by its lock ID (§6.6).

`vfxBlockHeight` is signed and emitted but not checked by the contract. Core signs set changes only for the current epoch boundary (§8.2).

6 Minting

Minting moves vBTC from VerifiedX to Base: the holder locks vBTC, the lock is buried, the contract's signers attest it, and `mintWithProof` mints the same number of satoshis to the EVM destination the lock names.

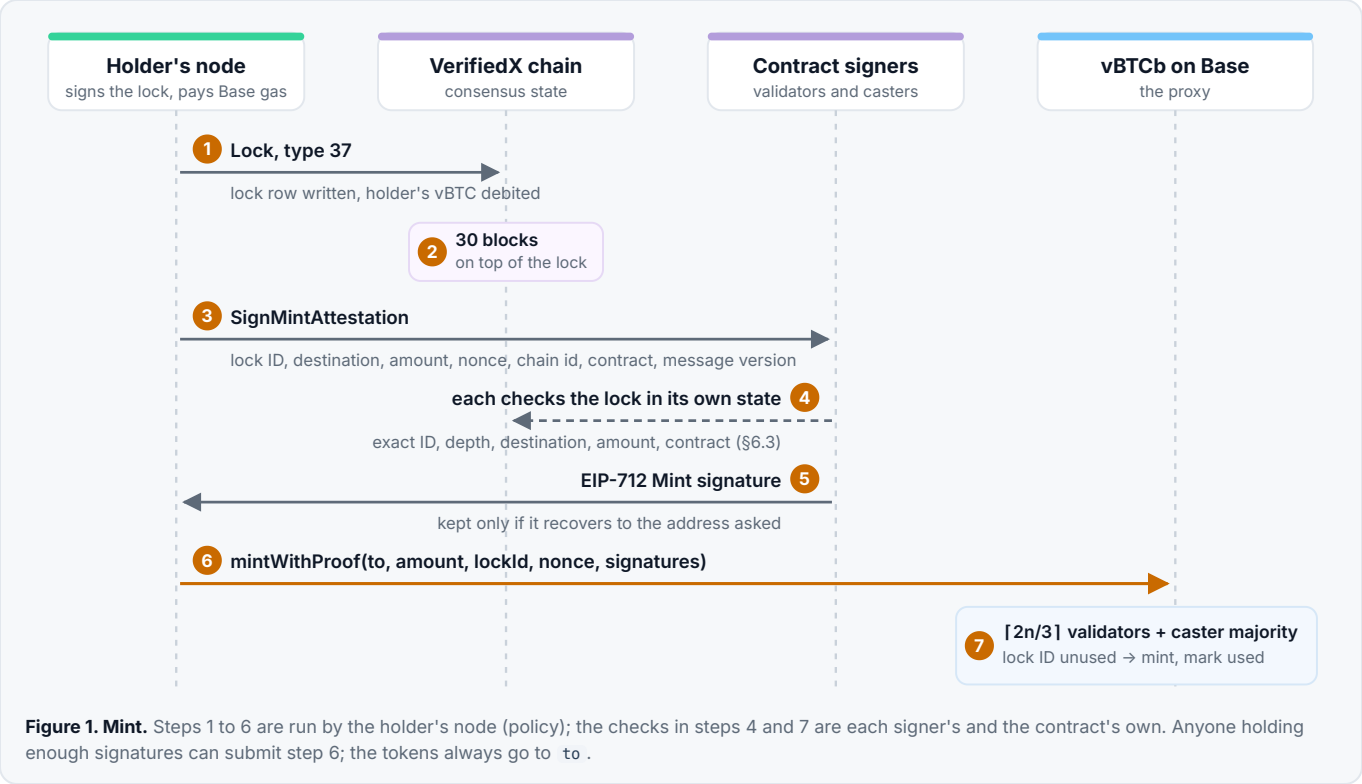


Figure 1. Mint. Steps 1 to 6 are run by the holder's node (policy); the checks in steps 4 and 7 are each signer's and the contract's own. Anyone holding enough signatures can submit step 6; the tokens always go to `to`.

6.1 Lock (type 37)

A bridge lock is a `VBTC_V2_BRIDGE_LOCK` transaction (type 37) with function `VBTCBridgeLock()`. It is valid only if:

- it is a self-transaction (sender equals recipient) from an address that is not a reserve account (`xRBX…`);
- its vault is a vBTC V2 contract, is not S3C, and has a key that can still sign;
- its `LockId` is not blank, has no surrounding spaces, is made only of letters, digits, "-" and "_", is at most 64 characters, and no lock with that ID exists on chain;
- its `Amount` is positive with at most 8 decimals, `AmountSats` equals `Amount × 10^8`, and the sender's vBTC balance on the vault covers it under the same rules as a transfer;
- its `EvmDestination` is `0x` and 40 characters, and does not begin with the bytes of "ADD", "REMOVE" or "UPGRADE" (validators never sign a mint to such an address, §6.3).

When the lock is applied, every node first writes the lock row (lock ID, vault, owner, amount, EVM destination, lock transaction hash, timestamp and block height) and then debits the owner's vBTC on the vault's ledger. A row already present is a replay and debits nothing. The vault's Bitcoin does not move. The lock joins the pool at once (§7.4), whether or not its mint has been submitted.

6.2 Collecting attestations (policy)

The holder's node waits for the lock to appear in its chain state and for 30 blocks on top of it. It then reads the contract: its validators, casters, `MESSAGE_VERSION()` and `requiredMintSignatures`. It asks every contract signer it can reach: registered vBTC validators, and caster committee members (including members of the 200 committee records before the current one), whose Base address is on either list. Each is asked at `valapi/Validator/SignMintAttestation` with the lock ID, destination, amount, nonce, chain id, contract address and message version. A returned signature is kept only if it recovers to the address asked. The node stops once both thresholds are met, retries up to three rounds, and then submits `mintWithProof` from the holder's derived Base address. It sets `nonce` to the lock's VFX block height.

6.3 What a signer checks before signing

A node signs a mint only if every check passes, in this order:

1. it runs as a validator;
2. the lock exists in its own consensus state;
3. the requested lock ID equals the stored ID byte for byte;
4. the lock's block has at least **30 blocks** on top (`MintAttestationDepth` = 3 × the deepest automatic reorganisation, 10). A row stored before lock heights were recorded is not held back;
5. the EVM destination equals the lock's (ignoring case) and does not begin with an admin tag (“ADD”, “REMOVE”, “UPGRADE”);
6. the amount in satoshis equals the lock's;
7. the contract address is the one this node is configured for and the chain id is its configured Base chain id;
8. the message version is 4, 3 or 0 (not sent);
9. the node is at the network's height, is not stalled, and does not see a majority of its peers on another chain.

For version 4 it signs the EIP-712 `Mint` digest over the destination, amount, **stored** lock ID, nonce, chain id and contract. For 3 or 0 it signs the V3 packed hash under the personal-message prefix (§9). The nonce is not checked, and the requester's identity is not checked: the signature only says that this lock, as stored, may be minted once.

6.4 Who signs

Any node running as a validator answers a well-formed request. The contract counts only signatures from addresses on its lists, so a signature from anyone else is skipped (§4.4). A signer that is on both lists contributes to both thresholds.

6.5 mintWithProof

The call must satisfy, in order:

1. `to` is not the zero address and `amount` is greater than zero;
2. `keccak256(lockId)` is not in `usedLockIds`;
3. the signatures over the `Mint` digest meet both thresholds (§4.4).

The contract then marks the lock ID used, mints `amount` to `to` and emits `MintExecuted(to, amount, lockId, nonce)`.

6.6 One mint per lock

Replay protection rests on the lock ID, not the nonce. The contract mints once per lock ID string. VerifiedX consensus allows one lock per ID, restricts IDs to one spelling (no surrounding spaces, a fixed character set), and signers sign only the stored ID exactly, so no variant spelling of the same lock can be minted again. Signatures with different nonces for the same lock can exist, but only the first mint of the lock ID succeeds. A lock is never returned to its holder except through a burn on Base: if the mint is not submitted, the holder's node can collect signatures and submit it later.

7 Exits

An exit starts with a burn on Base. The burn names either a VFX address, which is credited vBTC from the pool of locks, or a Bitcoin address, which is paid from the vaults those locks belong to.

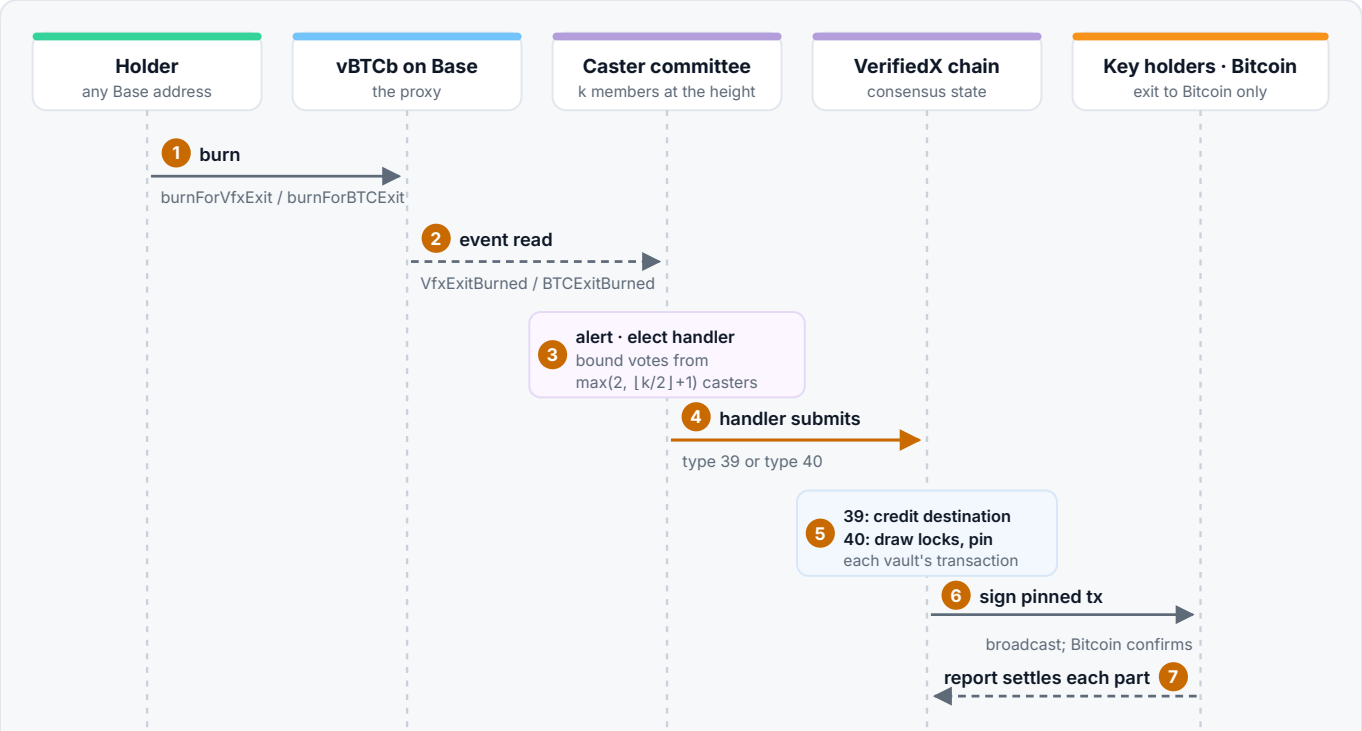


Figure 2. Exits. Steps 1 to 5 apply to both kinds. For an unlock (type 39) the destination is credited in step 5 and the exit is done. For an exit to Bitcoin (type 40) steps 6 and 7 follow; an exit not pinned within 360 blocks fails by rule (§7.8).

7.1 Burns on Base

`burnForVfxExit(amount, vfxDestinationAddress)` and `burnForBTCExit(amount, btcDestination)` burn from the caller and emit an event with the burner, amount in satoshis, destination string and `block.chainid`. Core names a burn by its Base transaction hash, or by `{hash}:{position}` when one transaction emits several bridge burns (position in emitted order, written without leading zeros). Each burn has one name, and once redeemed it is recorded as consumed on every path.

7.2 Caster agreement

Each caster polls Base every 12 seconds for the two events emitted by the configured proxy. For each burn (policy):

1. **Alert.** A caster that sees the burn signs `VFX_BURN_ALERT|burn|type|amountSats|destination|sender|timestamp` with its VFX key and sends it to the committee. An alert is accepted only from a committee member, within 600 seconds of the receiver's clock.
2. **Handler election.** Each caster proposes itself with the hash `keccak256("{proposer}:{burn}:{⌊timestamp/600⌋}")`. The lowest hash among committee proposers wins. Proposals are accepted within 120 seconds of the receiver's clock; a retry in a later 600-second window draws again, so no caster wins every round.
3. **Votes.** A caster that has read the burn from Base and found the amount and destination it expects signs the bound vote `VFX_BRIDGE_BURN_V2|burn|type|amountSats|destination|timestamp`, and a second signature naming the handler it agreed to.
4. **Submission.** The handler submits type 39 for a VFX exit (`POOL_EXIT`) or type 40 for a Bitcoin exit (`BTC_EXIT`), carrying the votes. The committee used is the one in force at the next block.

Consensus counts a vote only from a member of the caster committee for the block's height (the signed, hash-chained membership record for that height, or the network's seed list where no record covers it), only as a valid VFX signature over the bound message with this transaction's burn, type, amount and destination, and once per caster. At least `max(2, ⌊k/2⌋+1)` must count, and the sender must be a committee caster. A node also reads the burn's Base receipt when it admits the transaction to its mempool: the receipt must have succeeded and hold exactly that burn from the proxy, with the same amount and destination. Block validation does not read Base; it rests on the votes.

7.3 Unlock to VerifiedX (type 39)

A `VBTC_V2_BRIDGE_POOL_UNLOCK` (type 39, function `VBTCBridgePoolUnlock()`) is valid only if:

- it is a self-transaction from a committee caster, carries the votes of §7.2 for `POOL_EXIT`, and its burn has not been consumed on any path;
- `VfxDestinationAddress` is a valid VFX address with no surrounding spaces, and `TotalAmount` equals `TotalAmountSats / 10^8` exactly, with `TotalAmountSats > 0`;
- its `Allocations` are objects with exactly the fields `LockId`, `UnlockAmount` and `SmartContractUID`; each names an existing lock by its exact ID, once, not fully unlocked and not blacklisted, for a positive whole number of satoshis no larger than the lock's remainder; and they add up to the total exactly.

When it is applied, the burn is marked consumed first, then each lock is drawn down and the destination is credited that amount on the lock's vault ledger. The destination therefore receives vBTC on one or more vaults.

7.4 The lock pool

Every lock with a remainder, not blacklisted, is in the pool. Locks are fungible: any burn can draw on any lock, and a lock can be drawn in parts by several exits. The handler builds the allocation plan (policy): oldest lock first by lock timestamp, passing over locks under 0.0001 BTC unless no plan works without them, at most 100 locks and 24,000 bytes of allocations per plan. Consensus checks each allocation (§7.3, §7.5) but not the order. Within one block, a burn can be redeemed once and a lock drawn once.

7.5 Exit to Bitcoin (type 40)

A `VBTC_V2_BRIDGE_EXIT_TO_BTC` (type 40, function `VBTCBridgeExitToBTC()`) carries zero VFX and zero fee. It is valid only if:

- it is a self-transaction from a committee caster, carries the votes of §7.2 for `BTC_EXIT` bound to its total and Bitcoin destination, and its burn has not been used on any path;
- it carries allocations, all on vaults the chain watches (an exit never mixes watched and older vaults), each lock named once by its exact ID for no more than its remainder, in whole satoshis, adding up to the total exactly;
- its `BtcDestination` is a Bitcoin address on this network and is not the deposit address of any vault it draws on;
- each vault's part is at least **25,186 sats**: the fee of the smallest transaction (154 vB) at 160 sat/vB, plus a 546-sat output.

When it is applied, every node writes the exit record first (a burn that already has one draws nothing), draws each lock down by its allocation, and records one part per vault.

7.6 Pinning the payment

In the same block, every node computes each part's Bitcoin transaction from the vault's free coins in chain state, at **10 sat/vB**, paying the destination the part's amount less the fee (and less the part's share of any key-sweep shortfall the vault carries), in the canonical form vBTC V2 uses for withdrawals. The exit is pinned **on all its vaults in that block or on none**: a part is pinned only if its transaction can be built, the vault's key can still sign, and the same coins could bear its fee raised to 160 sat/vB. An exit with no pin has no signature anywhere and can fail as a whole; an exit with pins will be paid in full.

The vault's key holders sign only the pinned transaction for their vault's part, in a FROST session led by a committee caster or by a signer of that pinned transaction. No caster builds the Bitcoin transaction, reports its id, or completes or fails the exit.

7.7 Fees, settlement

Every 30 blocks, a pinned part whose newest transaction was pinned at least **2,160 blocks** earlier gets a replacement at twice the fee rate (10, 20, 40 … sat/vB) on the same coins, the larger fee taken out of the amount paid; at most 8 transactions per part and at most 5,000 sat/vB. Every node fixes the same replacement at the same block.

A part settles when a Bitcoin report verified by consensus shows one of its transactions. Its burn is then consumed for good, and the payment is recorded against the vault so that its owner's balance is not reduced a second time (the vBTC behind it was debited at the lock). The exit completes when every part has settled.

7.8 Failure, voiding and blacklisted locks

- **Not pinned in time.** An exit none of whose parts was pinned within **360 blocks** is failed by every node at the same block: its parts close, its locks are restored and stay in the pool, and its record is marked failed. The casters can then exit the same burn again from other locks.
- **A part voided, others paying.** If a coin pinned to a part is spent by another transaction, or vanishes in a Bitcoin reorganisation, while another part is pinned or paid, the voided part is fixed again on its vault's free coins once the voiding spend has **144 Bitcoin blocks** on it, so its earlier signed transaction cannot also confirm.
- **A part voided, nothing paying.** Otherwise the voided part's locks are restored and **blacklisted**: excluded from every later plan, permanently. The exit is marked failed and can be made again from other locks.

The caster-sent completion and failure transactions (types 41 and 42) are refused for exits on watched vaults; these exits end only by the rules above.

8 Governance

8.1 Admin nonce

Every admin message carries the contract's `adminNonce`. Each successful `addValidatorBatch`, `removeValidatorBatch`, `updateCasters`, `proposeUpgrade`, `proposeUpgradeDelay`, `proposeRemoval` and `cancelPending` increments it; the three `execute…` calls and mints do not. A signature set is therefore usable once, and only while no other admin action has happened since it was signed.

8.2 Validator set

Contract rules. `addValidatorBatch` needs a non-empty batch, a total of at most 50, the approval of both groups over `AddValidators`, and one signature from each new validator over the same digest, in the same order; each new address must be non-zero and not already a validator (a repeat in the batch fails this). `removeValidatorBatch` needs a non-empty batch that leaves at least 3 validators and the approval of both groups over `RemoveValidators`; each address must be a validator, once. Requiring the full approval to remove means no group short of two thirds can remove the others and become two thirds of what is left.

Core policy. Contract validators keep the list in step with VerifiedX once per epoch of **7,200 blocks**, acting on a boundary once it has 30 blocks on top:

- **Candidates** are registered public (not S3C) vBTC validators active at the boundary and holding the 5,000 VFX validator balance, each by the Base address of its own key.
- **Removals** are contract validators that exited, lost the balance, or sent no registration or heartbeat in the **14,400 blocks** before the boundary: at most $\min(n - 3, \lfloor (n - 1)/3 \rfloor)$ per epoch, one removal per epoch.
- **Additions** follow the removals: at most $\min(50 - r, \lfloor (r - 1)/2 \rfloor)$ for r remaining, so newcomers stay under a third of the set and it grows 3, 4, 5, 7, 10 … 50 over successive epochs. Newcomers are ordered by `keccak256(boundary block hash ‖ address)`, which is unknown when a validator registers.
- **Signing.** A node signs an `AddValidators` or `RemoveValidators` only if it is exactly the change it computes itself for the current, settled boundary at the contract's admin nonce, on a version 4 contract, from `BaseBridgeV4Height` (0 on testnet). Only a member of either list signs a removal; a newcomer signs its own addition. No validator change is signed while the contract has no casters.
- **Sending.** Contract validators take turns 50 blocks apart, in the same keccak order, to gather the signatures and send the change from their own Base address.

8.3 Caster set

Contract rules. `updateCasters(add, remove, …)` needs at least one address, the approval of both groups over `UpdateCasters`, and one signature from each added caster over the same digest. Removals are applied first (each must be a caster), then additions (non-zero, not already a caster); the result must hold 3 to 10 casters.

Core policy. A caster joins the contract once it was in the VerifiedX caster committee at the current epoch boundary and the two before, and leaves once it was out at all three; at most one joins and one leaves per epoch. A contract with no casters receives every member that was in at all three boundaries, up to 10, once there are at least 3. The caster change is sent before the validator change each epoch.

8.4 Upgrades

`proposeUpgrade(newImplementation, signatures)` needs an implementation address that holds code, no other upgrade pending (unless it has lapsed), and the approval of both groups over `ProposeUpgrade`. It records the implementation and an eta of `block.timestamp + upgradeDelay()`. From the eta, anyone may call `executeUpgrade()`, which clears the proposal and runs `upgradeToAndCall` with no call data. Holders who disagree have the delay to burn and exit.

8.5 Delay changes

`upgradeDelay()` is 48 hours until changed. `proposeUpgradeDelay(newDelay, …)` accepts 48 hours to 30 days, needs no other delay change pending (unless lapsed) and the approval of both groups, and waits the **current** delay before anyone can run `executeUpgradeDelay()`. The same delay governs upgrades, delay changes and removals.

8.6 Removing members that went dark

If so many validators or casters stop signing that the approval can no longer be reached, `proposeRemoval(oldValidators, oldCasters, …)` clears them. It needs at least one address; validators named must leave at least 3, casters named at least 3; each must be on its list, once; no removal may be pending unless lapsed. It needs a majority of the validators **and** a majority of the casters, **or** two thirds of the validators alone (for when the casters are the ones gone). After the delay, anyone may call `executeRemoval()`, which skips addresses already removed and checks the minimums again.

Core's operator route for this message (`SignBaseAdminAction`, answered on localhost only) refuses to sign a removal that names a validator still registered as an active vBTC validator or a caster still in the committee.

8.7 Cancellation

`cancelPending(signatures)` drops everything pending at once: an upgrade, a delay change and a removal. It needs something pending and the blocking minority of either group over `CancelPending`: `n − ⌈2n/3⌉ + 1` validators, or `c − (⌊c/2⌋+1) + 1` casters (no caster path while the contract has no casters). Those are the signers who could already refuse a mint, so cancellation gives them no power they lack. It increments the admin nonce.

Core policy: a member's node signs `CancelPending` only when a pending removal names a member it sees as live (a caster in the current committee, or a validator that keeps its seat at the boundary). Members take turns 10 minutes apart to gather the veto and send it. A removal that only clears dark members is left to run.

8.8 Grace period

A pending upgrade, delay change or removal can be executed from its eta until 14 days after it. After that it has lapsed: its execute call reverts, and a new proposal of the same kind may replace it. Approval given long ago therefore cannot be used after holders stopped watching.

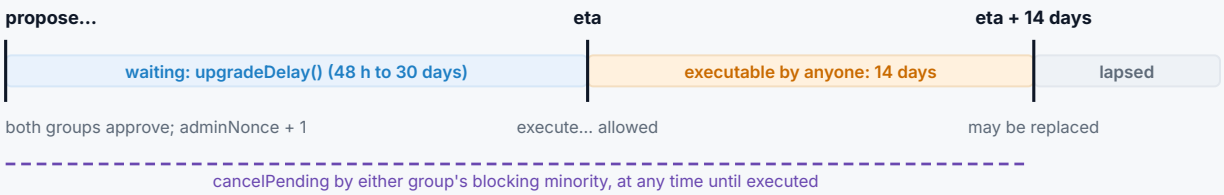


Figure 3. Life of a proposal. The same timeline applies to `proposeUpgrade`, `proposeUpgradeDelay` and `proposeRemoval`; one of each kind may be pending at a time.

8.9 How signatures are produced (Core)

ROUTE	WHAT THE NODE SIGNS
`valapi/Validator/SignMintAttestation`	A `Mint`, after the checks of §6.3. Open to any caller.
`valapi/Validator/SignValidatorUpdate`	`AddValidators`, `RemoveValidators`, `UpdateCasters` or `CancelPending`, only for the change the node computes itself (§8.2, §8.3, §8.7). Open to any caller.
`valapi/Validator/SignBaseAdminAction`	Localhost only, at the operator's request: `ProposeUpgrade` (returning the implementation's code hash for comparison with the reviewed build), `ProposeUpgradeDelay`, `ProposeRemoval` (§8.6), `CancelPending`, and on V3 the one `UPGRADE` message (§9). The node reads the contract's version and admin nonce itself.

9 Upgrading from V3

Base mainnet runs V3 behind the proxy `0x0909…132D`. The upgrade replaces the implementation in place: the proxy address, balances, used lock IDs, validators and admin nonce stay.

TOPIC	V3 (BASE MAINNET)	V4
Mint message	`keccak256(abi.encodePacked(to, amount, lockId, nonce, chainid, this))` under the personal-message prefix	EIP-712 `Mint` (§5)
Admin messages	Packed tags “ADD”, “ADD_BATCH”, “REMOVE”, “REMOVE_BATCH”, “UPGRADE”, same prefix	One EIP-712 type each
Signers	Validators only; at least 2	Validators (3 to 50) and casters (3 to 10)
Removal	`⌈n/2⌉` (at least 2) up to 5 validators; 51% above	Full approval, or the timelocked `proposeRemoval`
Upgrade	`upgradeWithValidatorApproval`: 2/3 of validators, immediate	Propose, wait, execute; cancellable; lapses
Bad signature	`ECDSA.recover` reverts the call	Skipped
Version	No `MESSAGE_VERSION()`; Core reads it as 3	`MESSAGE_VERSION()` = 4

Because V3's mint message begins with the destination's 20 bytes and its admin messages begin with an ASCII tag, a mint signature to a destination beginning with those bytes could be read as an admin message. Signers therefore never sign a mint to a destination beginning with “ADD”, “REMOVE” or “UPGRADE”, and consensus refuses a lock to one (§6.1). V4's typed messages make this unnecessary, but the rule stays.

The upgrade. Each V3 validator's operator signs the V3 `UPGRADE` message for the new implementation through `SignBaseAdminAction`; the signatures go to `upgradeWithValidatorApproval`, which upgrades at once with two thirds of the V3 validators. It is the last upgrade without a delay. Afterwards:

- the contract has **no casters**, so `requiredCasterSignatures()` is 0 and mints and changes need the validators alone, as on V3;
- Core adds the casters with one `updateCasters` (at least 3) from `BaseBridgeV4Height`, which is not yet set for mainnet, and makes no validator change before; from then on nothing passes without them, and the list cannot fall below 3;
- mints continue without a gap: nodes read `MESSAGE_VERSION()` and sign the form the contract reads;
- `requiredRemoveSignatures` may hold V3's value until the first validator change; V4 does not read it.

10 Transaction types

#	TYPE	SENT BY	PURPOSE
37	`VBTC_V2_BRIDGE_LOCK`	Holder	Lock vBTC for a mint on Base (§6.1).
38	`VBTC_V2_BRIDGE_UNLOCK`	None today	Legacy single-lock unlock from the first bridge version; current nodes send pool unlocks instead.
39	`VBTC_V2_BRIDGE_POOL_UNLOCK`	Handler caster	Credit a VFX address from the pool after `burnForVfxExit` (§7.3).
40	`VBTC_V2_BRIDGE_EXIT_TO_BTC`	Handler caster	Record an exit after `burnForBTCExit`; consensus pins its payment (§7.5).
41	`VBTC_V2_BRIDGE_EXIT_TO_BTC_COMPLETE`	Refused	For exits on watched vaults; Bitcoin reports settle them (§7.7).
42	`VBTC_V2_BRIDGE_EXIT_TO_BTC_FAIL`	Refused	For exits on watched vaults; they fail by rule (§7.8).

11 Parameters

PARAMETER	VALUE	WHERE
Validators on the contract	3 to 50	Contract
Casters on the contract	3 to 10	Contract
Upgrade delay	48 h to 30 days	Contract (default 48 h)
Proposal grace period	14 days	Contract
Token decimals	8	Contract
Lock depth before a mint is signed	30 blocks	Signer check
Lock ID length and characters	≤ 64; A–Z a–z 0–9 - _	Consensus
Caster votes for an unlock or exit	max(2, ⌊k/2⌋+1)	Consensus
Smallest vault part of an exit to Bitcoin	25,186 sats	Consensus
Exit fee rate; raise; cap	10 sat/vB; ×2; 5,000	Consensus
Unconfirmed blocks before a fee raise	2,160 blocks	Consensus (checked every 30)
Transactions per exit part	8	Consensus
Exit expiry without a pin	360 blocks	Consensus
Burial before a voided part is re-fixed	144 Bitcoin blocks	Consensus
Signer-set epoch; settle depth	7,200; 30 blocks	Core policy
Seat kept without a heartbeat	14,400 blocks	Core policy
Submit turns; cancel turns	50 blocks; 10 min	Core policy
Alert skew; proposal skew; election window	600 s; 120 s; 600 s	Core policy
Locks per plan; dust passed over	100; 0.0001 BTC	Core policy

12 Security model

12.1 Trust assumptions

- **Issuance.** No mint without a lock unless at least `⌈2n/3⌉` contract validators and `⌊c/2⌋+1` contract casters collude (or are compromised). Keys on both lists count for both groups (§4.4).
- **Lock finality.** Signers attest a lock only at 30 blocks deep and only while on the network's chain. A VerifiedX reorganisation deeper than that is not bounded by the depth rule.
- **Exits.** A majority of the caster committee for the height authorizes every unlock and exit. Block validation rests on those votes, so holders rely on a committee majority to report Base burns faithfully, as they rely on the validators for mints.
- **Bitcoin payment.** A threshold of each vault's key holders signs the pinned transaction (see the vBTC V2 Technical Specification). A colluding threshold holds the vault key's full signing power.
- **Governance.** A code change needs both groups and waits at least 48 hours, during which either group's blocking minority can cancel it and holders can exit.
- **Base.** Base orders and executes the contract's calls; the VerifiedX side reads Base through RPC endpoints the node operator configures.

12.2 Attacks and mitigations

ATTACK	MITIGATION
Signature for one action used as another	One EIP-712 type hash per action (§5.3); on V3, no mint to an admin-tag destination (§9).
Replay on another chain or contract	Domain binds chain id and proxy; signers sign only for their configured contract and chain.
Second mint of a lock	`usedLockIds`; one lock per ID on VerifiedX; signers sign the stored ID exactly.
Mint against a lock a reorganisation drops	30-block depth; no signing off the network's chain.
One signer counted many times	Each recovered address counted once per group.
Malformed signature blocking a call	Unrecoverable signatures are skipped.
Seats for keys nobody holds	Each new validator and caster signs its own addition.
A faction removing the rest	Removal needs the mint approval; the majority path is timelocked and cancellable; Core vetoes removals of live members.
Mass registration to reach two thirds	Core adds fewer than a third per epoch, ordered by a boundary hash unknown at registration.
Taking a caster seat briefly	A caster joins the contract only after three boundaries in the committee; one change per epoch.
Malicious upgrade	Both groups, a delay of 48 hours or more, cancellation, lapse; Core returns the implementation's code hash.
Redeeming a burn twice	Consumed-burn record on every path; one redemption per burn per block; one name per burn.
Exit for the wrong amount or address	Votes bind amount and destination; mempool admission checks the Base event.
Handler that never acts	Election redrawn every 600-second window; an unpinned exit fails by rule and can be made again.
Forged failure to blacklist locks	Types 41 and 42 refused for watched exits; blacklisting happens only by rule (§7.8).
Exit stuck at a low fee	Fee doubled by rule; each part sized to bear four doublings.
Paying a voided part twice	Re-fixed only after the voiding spend has 144 Bitcoin blocks on it.

12.3 Signature handling summary

On Base: 65-byte secp256k1 signatures over EIP-712 digests, low-*s* only (OpenZeppelin `ECDSA.tryRecover`), deduplicated by recovered address, foreign and malformed ones skipped, counted per group. On VerifiedX: caster votes are VFX signatures over text messages that bind burn, type, amount, destination and timestamp, counted once per committee member.

12.4 Limits integrators must observe

- **Destinations are not checked on Base.** A `burnForVfxExit` to an invalid VFX address, or a `burnForBTCExit` to a string that is not a Bitcoin address on the network, burns tokens that are never unlocked or paid.
- **Bitcoin exits have a minimum.** Each vault's part must be at least 25,186 sats, so a Bitcoin exit smaller than that cannot be paid. The contract does not refuse such a burn.
- **Vault deposit addresses.** An exit is never paid from the vault whose deposit address it names.
- **The mint nonce carries no protection.** Use the lock ID to track mints.

13 Deployed addresses

NETWORK	VERSION	ADDRESSES
Base Sepolia (84532)	V4	Proxy `0x3669EE313f9Aa14b9eBb787DfdB0E97Ca67A0510` Implementation `0xa06e3B68B495e474B41595435675f94CF8e7C711`
Base mainnet (8453)	V3 until upgraded	Proxy `0x09096c6eBf35F0aDAEeC02D06a0dE77E790F132D` V3 implementation `0xd2bB4BdA89A23c63eaaD464F15658845a2114Ffa`

Nodes and users always use the proxy. A node reads the contract and chain from `BaseBridgeContract` and `BaseBridgeChainId` in its `config.txt`. The V4 source is `deploy/contracts/vBTCb.sol` in VerifiedX Core; its flattened form `deploy/VBTCbV4.flattened.sol` is compiled with solc 0.8.28, optimizer on with 200 runs, EVM version cancun.

14 Glossary and references

Lock	A type 37 transaction and the consensus row it writes: vBTC taken from its holder to back vBTC.b.
Lock ID	The lock's unique name on VerifiedX and its replay key on Base.
Pool · allocation	All locks with a remainder that are not blacklisted · one lock and the amount an unlock or exit draws from it.
Blacklisted lock	A lock excluded from every future plan after an exit part on it was voided with nothing else paying.
Burn ID	A Base transaction hash, or `{hash}:{position}` for one of several burns in it.
Caster committee	The VerifiedX casters named by the signed membership record for a height.
Handler · bound vote	The committee caster elected for a burn · a caster's signature over a burn's ID, type, amount, destination and time.
Part · pin	One vault's share of an exit to Bitcoin · the Bitcoin transaction consensus fixed for it.
Watched vault	A vBTC V2 vault whose Bitcoin consensus tracks from verified reports.
Blocking minority	The fewest members of a group whose refusal prevents a mint; the signers `cancelPending` needs.
Admin nonce	The counter every admin message carries; each admin action increments it.
Epoch boundary	A VFX height divisible by 7,200 at which Core computes signer-list changes.

Contract	VerifiedX Core `deploy/contracts/vBTCb.sol` (V4), `deploy/contracts/legacy/vBTCbV3.sol` (V3), `deploy/README.md`
Core	`Bitcoin/Services/BaseBridgeAttestationService.cs`, `VbtcbTypedData.cs`, `BaseSignerSetPolicy.cs`, `BaseValidatorSyncService.cs`, `BridgeCasterConsensus.cs`, `Bitcoin/Attestation/BtcWithdrawals.Exits.cs`, `Services/TransactionValidatorService.cs`
Companion documents	vBTC V2 Technical Specification; docs.verifiedx.io: vBTC protocol › vBTC.b on Base; Integration › vBTC › Bridging to Base
Standards	EIP-712 (typed data), EIP-191, ERC-20, ERC-1967, ERC-1822 (UUPS), BIP 125

This document describes protocol behaviour as implemented in VerifiedX Core on testnet at commit ee83f568 and in the vBTCb V4 contract. It is not an offer of any asset or a statement of any asset's legal or regulatory status. Where this document and the source differ, the source governs.